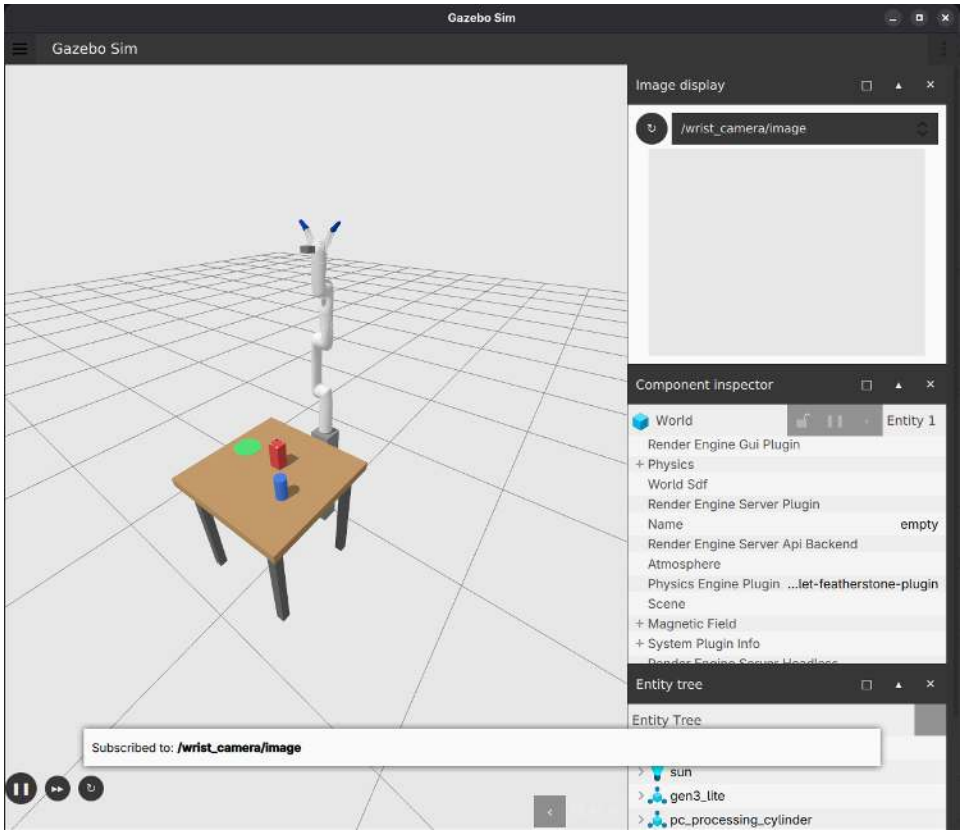
Group Assignment Report

Overview

For this project we built a pipeline that looks at objects on a table with a depth camera and figures out how to pick one up. It combines several camera views into one point cloud, removes the table, splits the rest into objects, finds a stable two-finger grasp, and has the robot pick the object up. We developed it in simulation (Part 1) and then ran the same code on the real Kinova Gen3 lite with a wrist-mounted RealSense D405 (Part 2).

The code is split into two ROS2 packages: `cvproc` (C++/PCL) handles the vision and grasp planning (Parts 1.1-1.3), and `kinova_sim` (Python/MoveIt) handles the simulation, moving the camera, and executing the grasp (Part 1.4). All the tunable settings are in YAML config files, so switching to the real robot only means swapping configs.

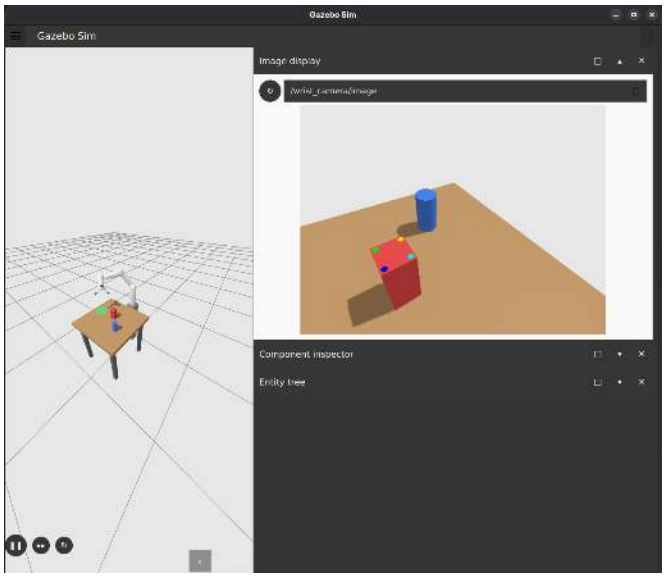


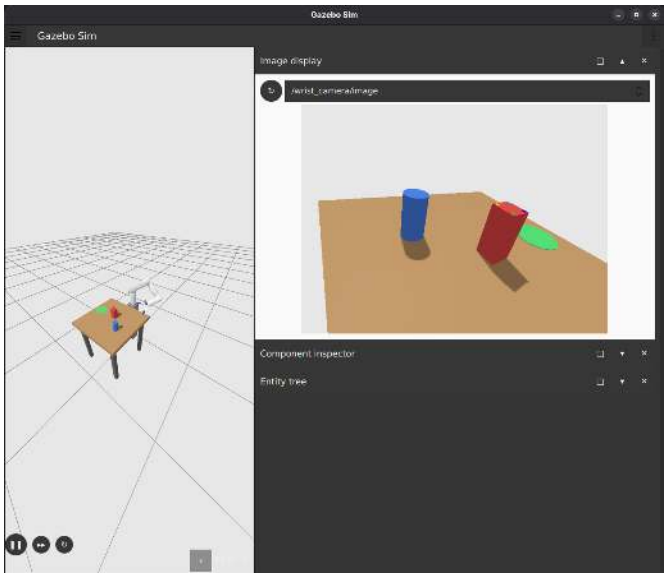
Part 1: Simulation Implementation

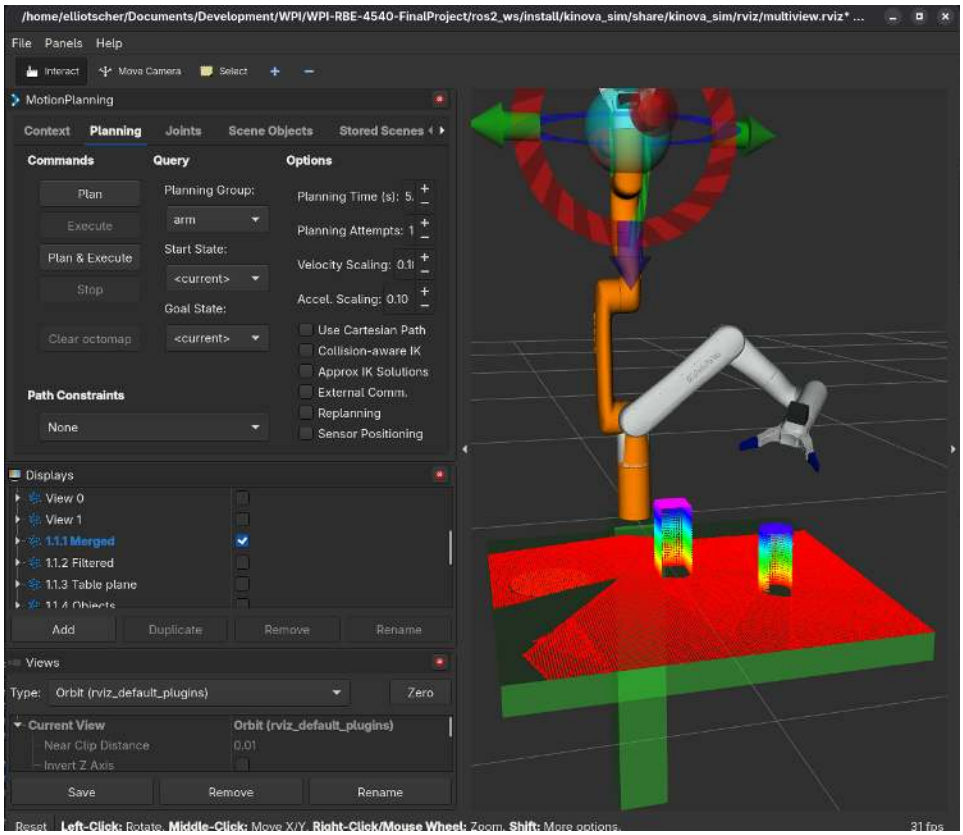
Part 1.1: Point Cloud Preprocessing

Part 1.1.1: Merge the Point Clouds

The arm moves the wrist camera to two positions on opposite sides of the objects and takes a point cloud at each. Both clouds are transformed into the robot's base frame so they line up, combined, and downsampled to remove the duplicate points where the views overlap.







Relevant code (`point_cloud_merge.cpp`):

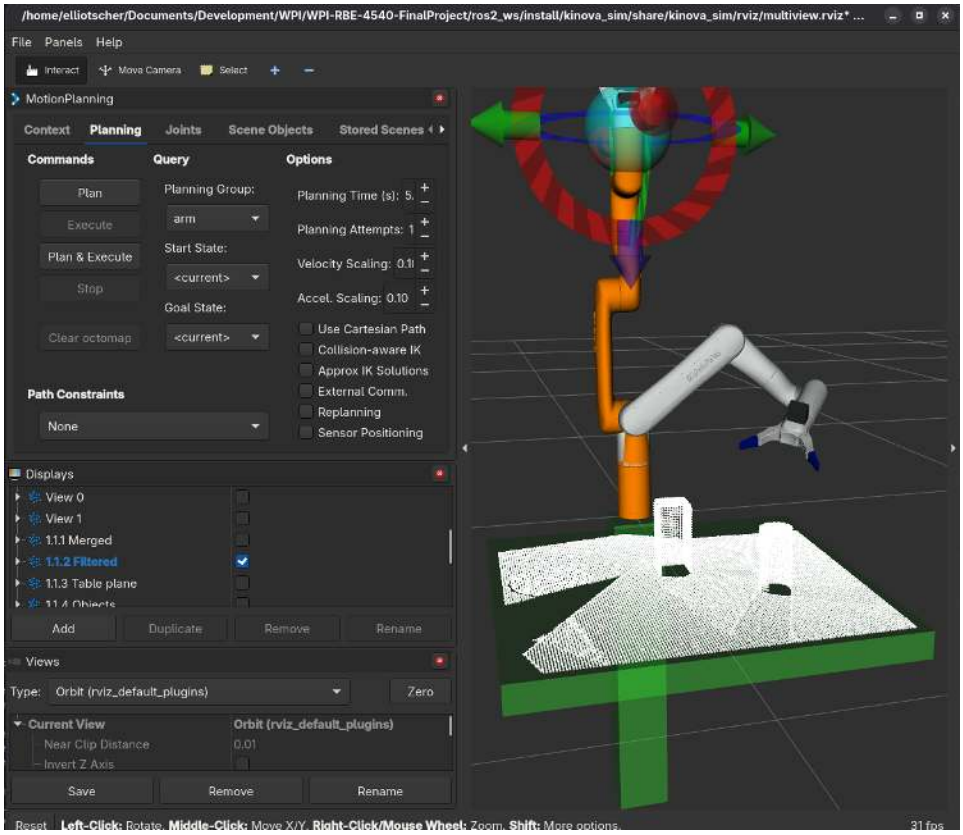
```
Cloud::Ptr cloudInFrame(const sensor_msgs::msg::PointCloud2& msg, const tf2_ros::Buffer&
        tf_buffer, const std::string& target_frame, const rclcpp::Duration& timeout) {
  const auto tf = tf_buffer.lookupTransform(
    target_frame, msg.header.frame_id, rclcpp::Time(msg.header.stamp), timeout);
  return transformCloud(*cloudFromMsg(msg), tf2::transformToEigen(tf));
}

Cloud::Ptr mergeCloudMsgs(const std::vector<sensor_msgs::msg::PointCloud2::ConstSharedPtr>&
        msgs, const tf2_ros::Buffer& tf_buffer, const std::string& target_frame, float
        leaf_size, const rclcpp::Duration& timeout) {
  Cloud::Ptr merged(new Cloud);
  for (const auto& msg : msgs) {
    if (msg) {
      *merged += *cloudInFrame(*msg, tf_buffer, target_frame, timeout);
    }
  }
  return downsampleIfRequested(merged, leaf_size);
}
```

Stage	Points
View 0	150512
View 1	154229
Merged + downsampled	22981

Part 1.1.2: Filter the Workspace

The camera also picks up the floor, the table legs and the background. We cut out a box around the table top and throw away everything outside it.

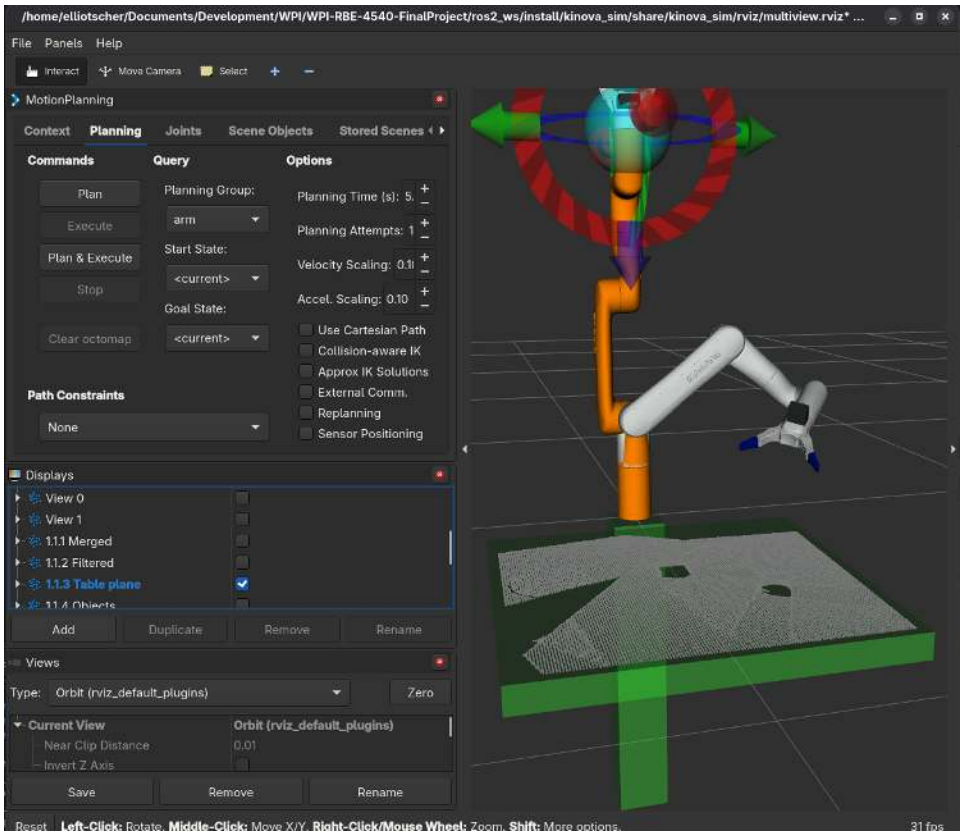


Relevant code (`filter_point_cloud.cpp`):

```
pcl::CropBox<pcl::PointXYZ> crop;
crop.setInputCloud(input_cloud);
// CropBox uses homogeneous coordinates, so the 4th value is always 1.0.
crop.setMin(Eigen::Vector4f(params.x_min, params.y_min, params.z_min, 1.0f));
crop.setMax(Eigen::Vector4f(params.x_max, params.y_max, params.z_max, 1.0f));
crop.filter(*output_cloud);
```

Part 1.1.3: Segment the Table Plane

We find the table by fitting a flat plane to the cloud with RANSAC. The plane is also used later as the table height and the "up" direction.



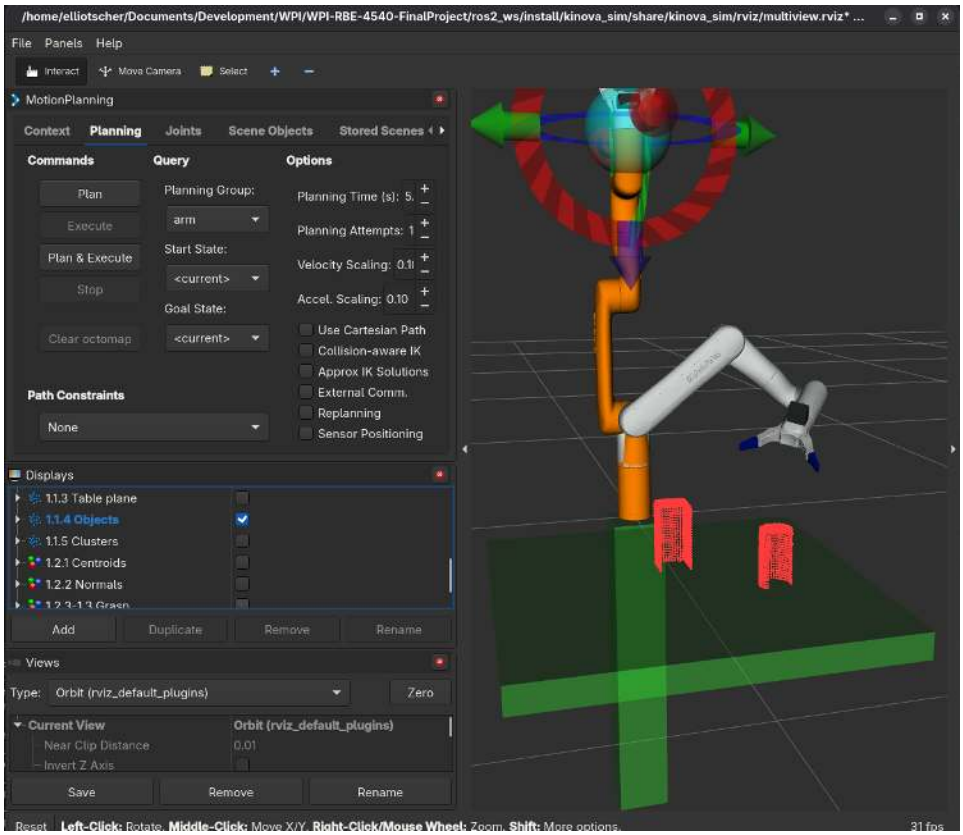
Fitted plane: `0.000x + 0.001y + 1.000z + -0.000 = 0`

Relevant code (`segment_table_plane.cpp`):

```
pcl::SACSegmentation<PointT> segmentation;
segmentation.setOptimizeCoefficients(true);
segmentation.setModelType(pcl::SACMODEL_PLANE);
segmentation.setMethodType(pcl::SAC_RANSAC);
segmentation.setDistanceThreshold(distance_threshold_);
segmentation.setMaxIterations(max_iterations_);
segmentation.setInputCloud(cloud);
segmentation.segment(inliers, coefficients);
```

Part 1.1.4: Segment the Objects

We remove the points that belong to the table plane, which leaves just the objects.

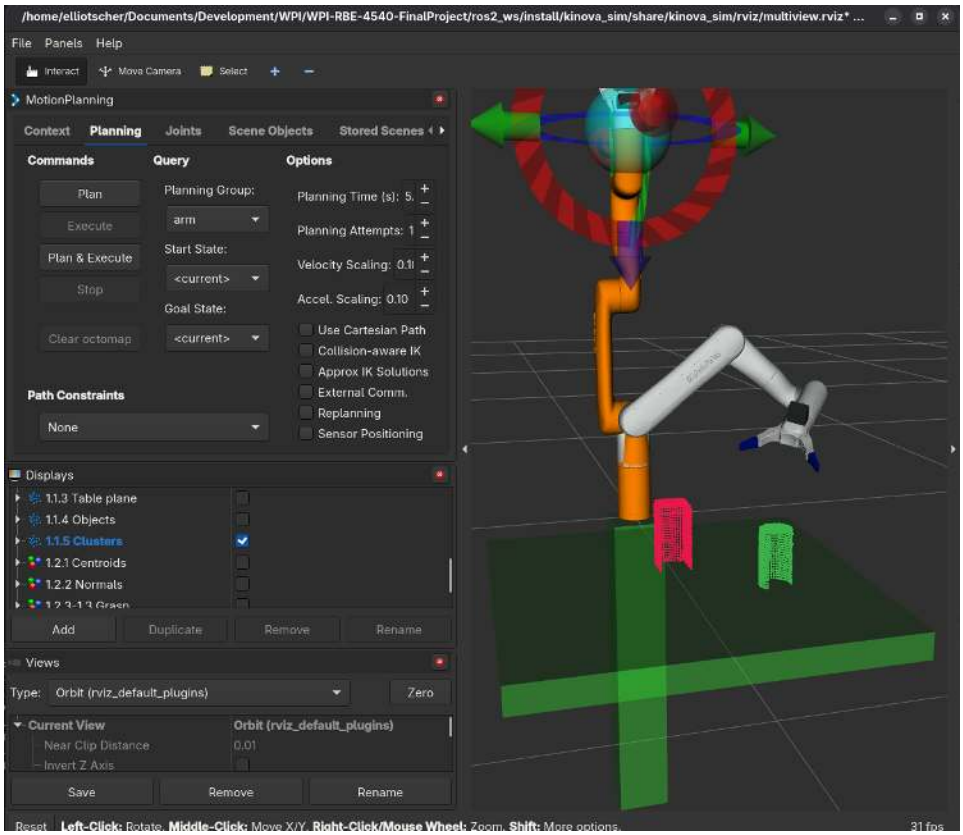


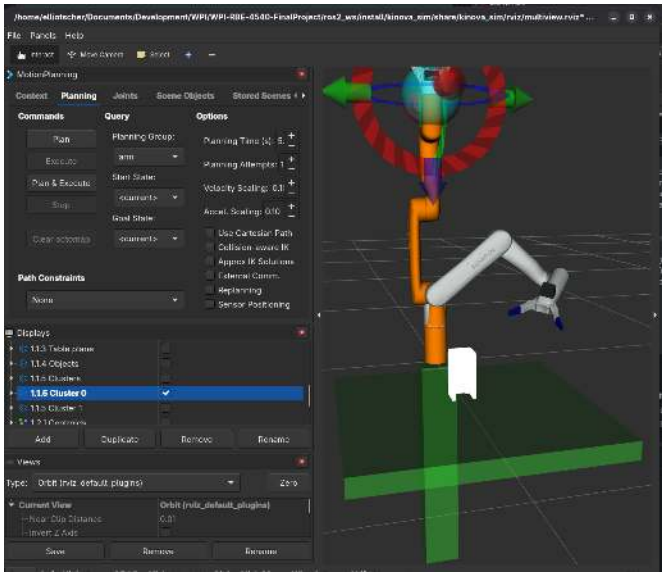
Relevant code (`pointcloud_pipeline_node.cpp`):

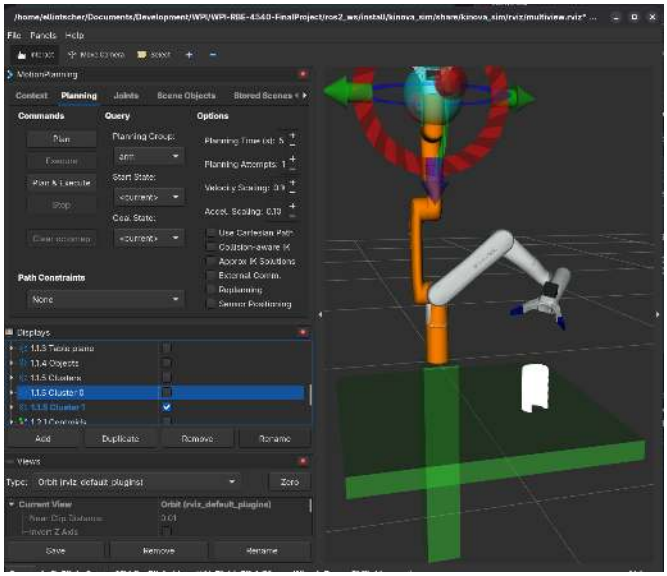
```
auto table = TablePlaneSegmentor::extract(filtered, inliers, false);
auto objects = TablePlaneSegmentor::extract(filtered, inliers, true);
if (object_min_height_ > 0.0 && plane.values.size() == 4) {
  Eigen::Vector4f c(plane.values[0], plane.values[1], plane.values[2], plane.values[3]);
  c /= c.head<3>().norm();
  if (c.z() < 0.0f) {
    c = -c;  // height = n.p + d, positive above the table (base_link z is up)
  }
  auto above = pcl::make_shared<Cloud>();
  for (const auto & p : objects->points) {
    if (c.head<3>().dot(p.getVector3fMap()) + c[3] > object_min_height_) {
      above->push_back(p);
    }
  }
  objects = above;
}
```

Part 1.1.5: Cluster the Objects

We group the remaining points into separate objects: points close to each other belong to the same object. Tiny groups are thrown out as noise.







Cluster	Object	Points
0	Cube	1131
1	Cylinder	867

Relevant code (`cluster_objects.cpp`):

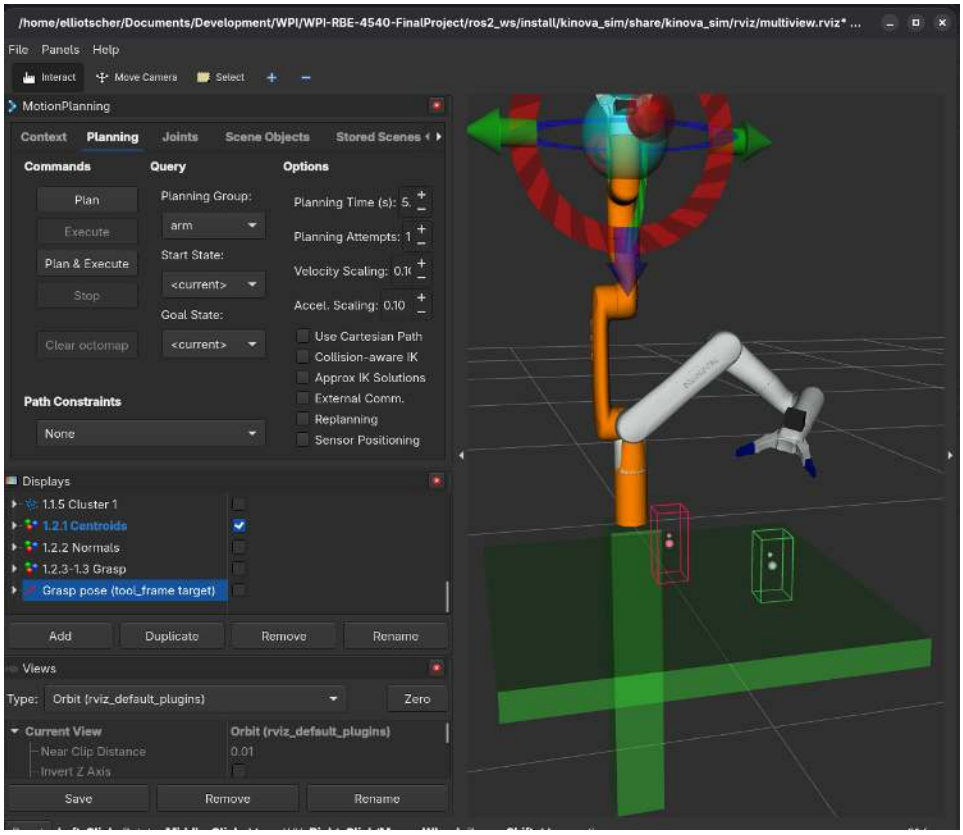
```cpp
auto tree = pcl::make_shared<pcl::search::KdTree<pcl::PointXYZ>>();
tree->setInputCloud(cloud);

std::vector<pcl::PointIndices> cluster_indices;
pcl::EuclideanClusterExtraction<pcl::PointXYZ> ec;
ec.setClusterTolerance(params.tolerance);
ec.setMinClusterSize(params.min_size);
ec.setMaxClusterSize(params.max_size);
ec.setSearchMethod(tree);
ec.setInputCloud(cloud);
ec.extract(cluster_indices);  // sorted largest first
```

Part 1.2: Grasp Synthesis

Part 1.2.1: Calculate the Object Centroid

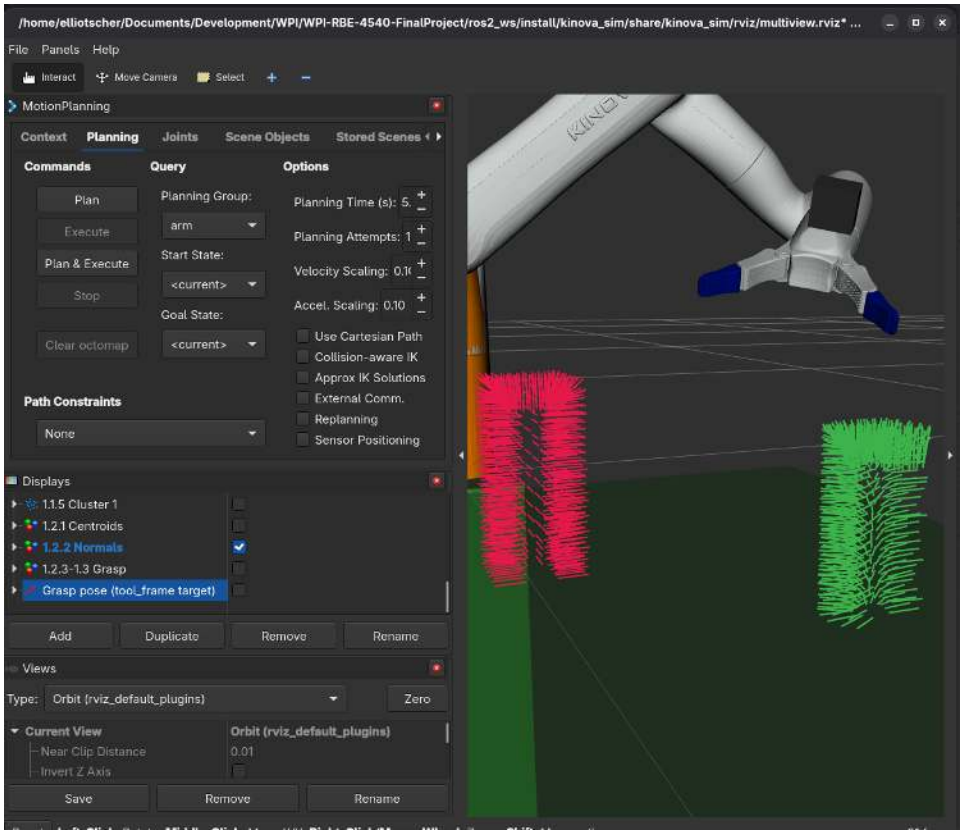
Since the camera only sees the top and some sides of each object, simply averaging its points gives a center that's too high and off to one side. Instead, we use the center of the object's footprint on the table, halfway up its height.



Cluster	Mean (m)	Footprint center (m)	Footprint box (m)
0	(0.392, 0.001, 0.084)	(0.399, 0.000, 0.070)	0.060 x 0.059 x 0.140
1	(0.542, 0.158, 0.079)	(0.549, 0.160, 0.060)	0.060 x 0.057 x 0.120

Part 1.2.2: Calculate Surface Normals

We estimate the surface normal (the direction the surface faces) at every point on each object, and flip them all to point into the object.



Relevant code (`surface_normals.cpp`):

```
pcl::NormalEstimation<pcl::PointXYZ, pcl::Normal> ne;
ne.setInputCloud(cloud);
ne.setSearchMethod(pcl::make_shared<pcl::search::KdTree<pcl::PointXYZ>>());
ne.setRadiusSearch(params.radius);
// Flips each normal so that n . (centroid - p) >= 0.
ne.setViewPoint(centroid.x(), centroid.y(), centroid.z());
ne.compute(*normals);
```

Part 1.2.3: Search for Feasible Contact Pairs

We test pairs of points on each object as places for the two fingers. A pair is kept if both surfaces face roughly toward each other along the line between the points (inside the friction cone), the points fit inside the gripper, and the line is level enough for the fingers to come down around the object.

Relevant code (`grasp_selection.cpp`):

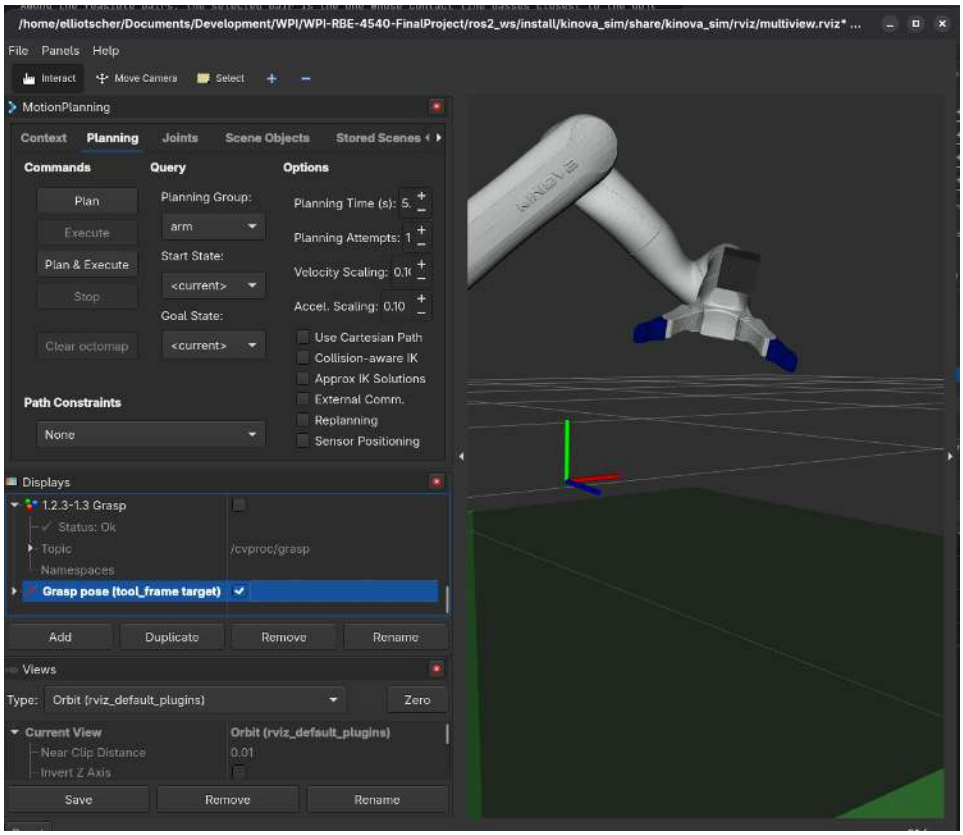
```
// 1. L nearly level, so the open fingers straddle the object on the way down.
const float up_dot = std::abs(up_unit.dot(l));
if (up_dot > max_up_dot) {
  continue;
}
// 2. Width fits the gripper.
if (width < params.min_width || width > params.max_width) {
  continue;
}
// 3. Friction cone at each contact (cheap dot-product test before any acos).
const Eigen::Vector3f n2 = normals.points[j].getNormalVector3fMap();
const float c1 = n1.dot(l);
const float c2 = -n2.dot(l);
if (c1 < cos_max || c2 < cos_max) {
  continue;
}
// 4. L far enough from the major axis for a well-defined orthogonal approach.
if (std::abs(major.dot(l)) > max_major_dot) {
  continue;
}
++st.feasible;
```

Cluster	Points searched	Pairs tested	Feasible pairs
0	1131	639015	14255
1	867	375411	8210

Part 1.2.4: Select the Grasp Pair

Many pairs pass, so we score each one by how close it is to a point a little above the object's center and how well its normals line up, and take the lowest score. The other passing grasps are kept as backups, sorted by score, in case the arm can't reach the best one.

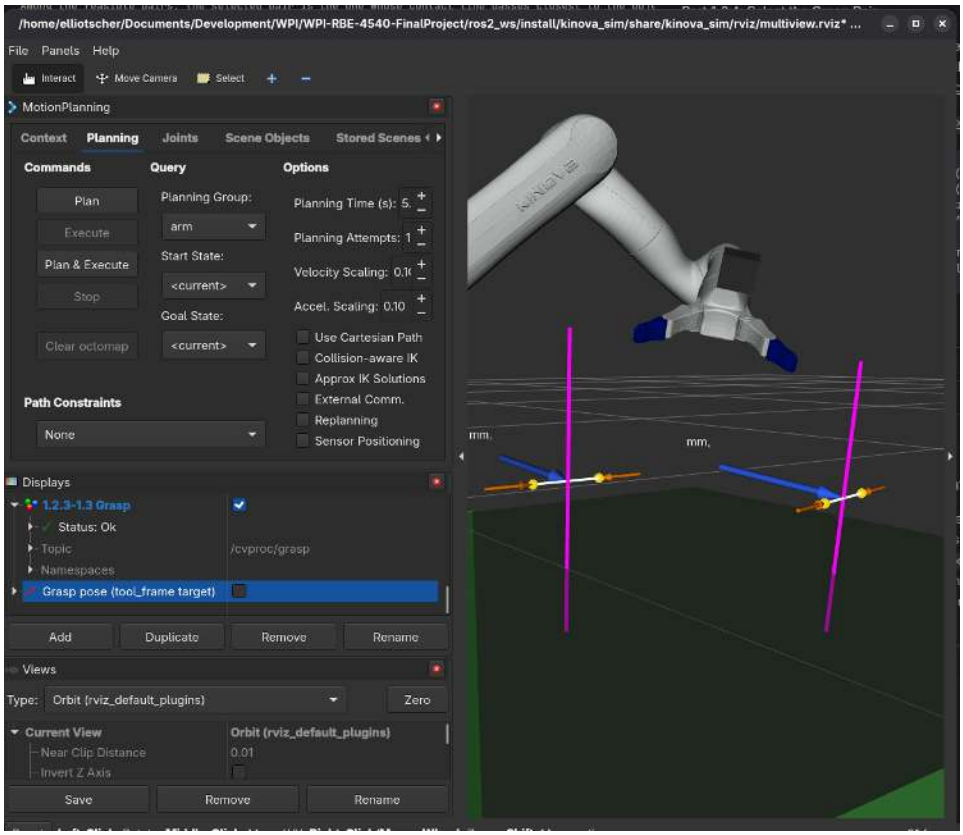
```
const float angle1 = std::acos(std::min(c1, 1.0f));
const float angle2 = std::acos(std::min(c2, 1.0f));
const float t = std::clamp((centroid - p1).dot(l), 0.0f, width);
const float dist = (centroid - (p1 + t * l)).norm();
const float cost = static_cast<float>(
  w_angle * (angle1 + angle2) / (2.0 * max_angle) +
  (1.0 - w_angle) * dist / distance_scale);
```



Cluster	theta1 (deg)	theta2 (deg)	Centroid distance (mm)	Width (mm)	Cost
0	0.1	0.1	2.8	60.0	0.072
1	1.7	0.5	2.5	59.7	0.088

Part 1.3: Calculate the Grasp Orientation (Approach Vector)

We use PCA to find the object's long axis. The gripper comes in perpendicular to both the long axis and the line between the fingers, usually from above. Since the arm can't always reach that exact angle, we also try a few tilted versions of each grasp, scored slightly worse so the original direction wins when it's reachable.



Relevant code (`principal_axes.cpp`, `grasp_selection.cpp`):

```
Eigen::Matrix3f covariance;
Eigen::Vector4f mean;
pcl::computeMeanAndCovarianceMatrix(cloud, covariance, mean);

// Eigenvalues come out in increasing order: column 2 is the major axis.
const Eigen::SelfAdjointEigenSolver<Eigen::Matrix3f> solver(covariance);
```

```
// The only direction orthogonal to both the major axis and the closing axis.
Eigen::Vector3f a = major_axis.cross(closing);
a.normalize();

const Eigen::Vector3f up_unit = up.normalized();
if (std::abs(a.dot(up_unit)) > std::sin(horizontal_tolerance)) {
  // Tilted: come from above.
  return a.dot(up_unit) < 0.0f ? a : Eigen::Vector3f(-a);
}
// Horizontal: come from the robot's side, i.e. pointing away from the base.
Eigen::Vector3f outward = center - robot_origin;
outward -= outward.dot(up_unit) * up_unit;
return a.dot(outward) >= 0.0f ? a : Eigen::Vector3f(-a);
```

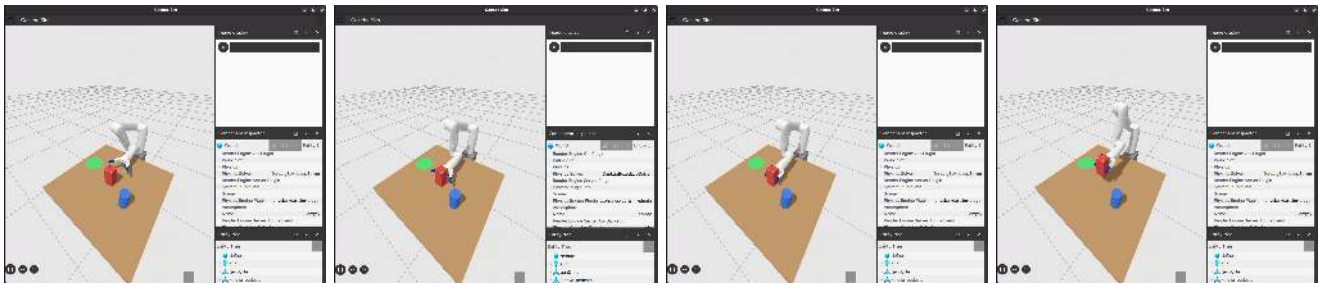
```
const Eigen::Vector3f side = g.closing.cross(*approach);  // in the approach plane, normal to L
for (const double tilt : tilts) {
  Grasp v = g;
  v.tilt = static_cast<float>(tilt);
  v.approach = (std::cos(v.tilt) * *approach + std::sin(v.tilt) * side).normalized();
```

```
  const float down_dot = -v.approach.dot(up_unit);  // sin(angle below horizontal)
  if (down_dot < min_down_dot || down_dot > max_down_dot + 1e-6f) {
    continue;  // too flat (or from below), or outside the allowed range
  }
  v.cost = pair.cost + static_cast<float>(params.approach_tilt_weight * std::abs(tilt) / (M_PI
        / 2.0));
  // ... skip near-duplicates, build the orientation (x = closing, z = approach)
}
```

Cluster	Major axis	Approach	Grasp center (m)
0	(0.06, -0.00, 1.00)	(1.00, 0.00, -0.06)	(0.397, 0.000, 0.072)
1	(0.13, 0.02, 0.99)	(0.96, 0.26, -0.13)	(0.550, 0.160, 0.062)

Part 1.4: Execute the Grasp

Once the camera views are done and the grasps arrive, the executor checks which grasps the arm can actually reach without collisions, starting from the best. It then carries out the best reachable one: open the gripper, move to a pre-grasp pose 10 cm back, move straight in, close the gripper, and lift straight up.



Relevant code (`grasp_executor.py`):

```
def check_grasp(self, T_tool, ee):
    T_pre = T_tool.copy()
    T_pre[:3, 3] -= self.pregrasp_offset * T_tool[:3, 2]
    T_lift = T_tool.copy()
    T_lift[2, 3] += self.lift_height
    pre = self.mover.solve_ik(ee(T_pre))
    if pre is None:
        return None, 'no collision-free IK at the pre-grasp'
    approach = self.mover.plan_cartesian(ee(T_tool), start=pre, quiet=True)
    if approach is None:
        return None, 'approach not a feasible straight line'
    end = approach.joint_trajectory
    at_grasp = dict(zip(end.joint_names, end.points[-1].positions))
    if self.mover.plan_cartesian(ee(T_lift), start=at_grasp, quiet=True) is None:
        return None, 'lift not a feasible straight line'
    return pre, ''
```

```
# 1. kinematic feasibility first: keep only the grasps the arm can do.
feasible = self.feasible_grasps(grasp.poses, ee, T_tool_cam)
if not feasible:
    log.error('No kinematically feasible grasp; not moving')
    return
# 2. selection among the feasible grasps only: lowest cost (the list keeps cvproc's order).
k, T_tool, joints = feasible[0]
T_lift = T_tool.copy()
T_lift[2, 3] += self.lift_height

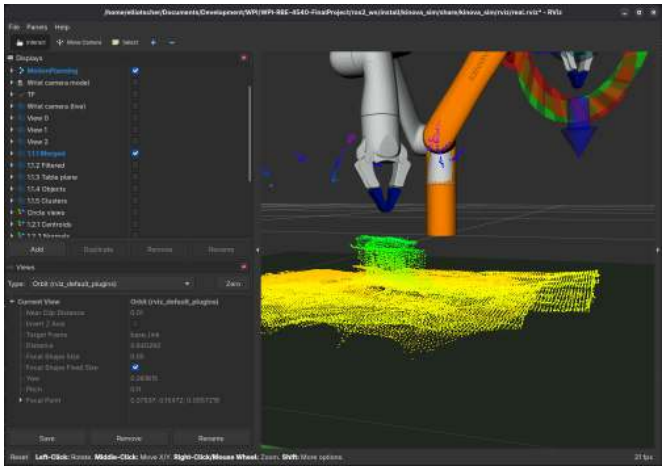
if not self.set_gripper(self.open_position, 'open'):
    return
if not self.mover.move_to_joints(joints):
    return
if not self.mover.move_cartesian(ee(T_tool)):
    return
if not self.set_gripper(self.close_position, 'close'):
    return
if not self.mover.move_cartesian(ee(T_lift)):
    return
```

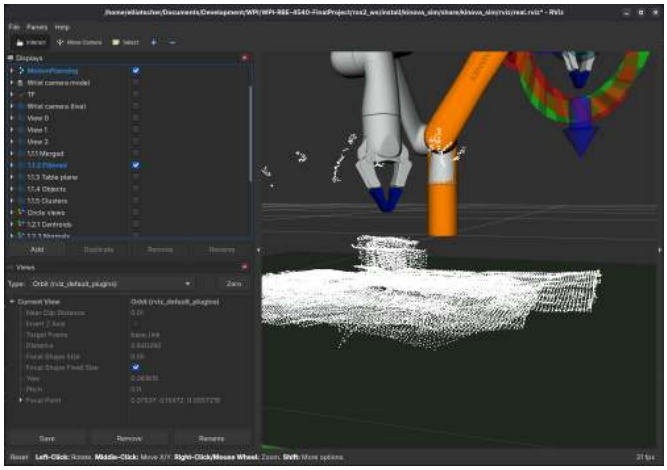
A video of the simulated robot moving to the pre-grasp pose, approaching, grasping and lifting the object (`sim_grasp.webm`), and the zipped `cvproc` and `kinova_sim` packages, are submitted alongside this report.

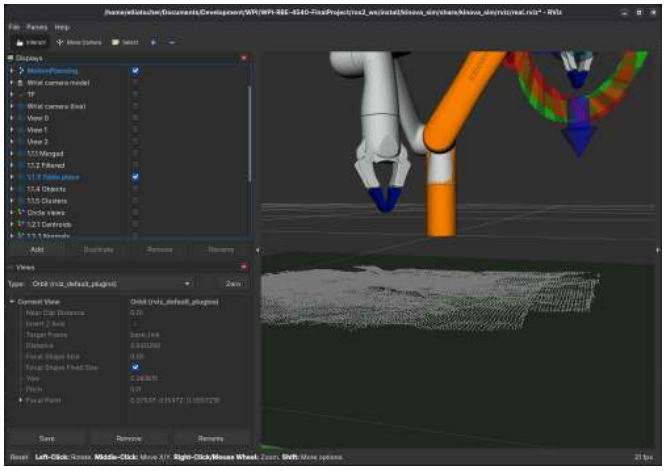
Part 2: Real-Robot Implementation

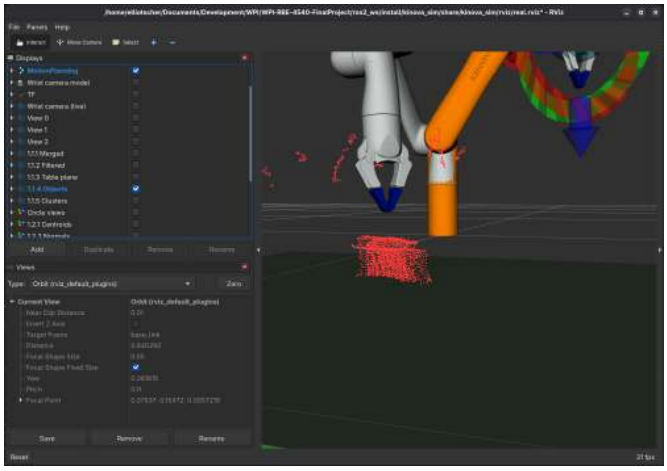
On the real robot we run the same nodes with `real_pipeline.launch.py`, which also starts the arm and camera drivers. Only the settings change: the arm uses 4 camera views instead of 2, the workspace box and table height match the lab table, and the object step is a bit stricter, since the real views don't line up perfectly and leave extra layers of table points behind.

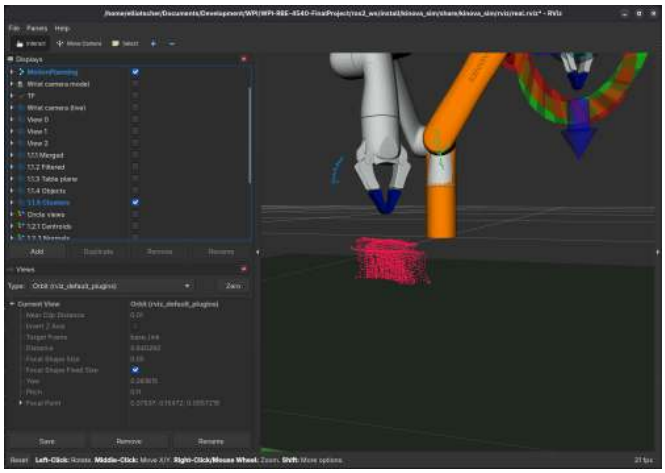
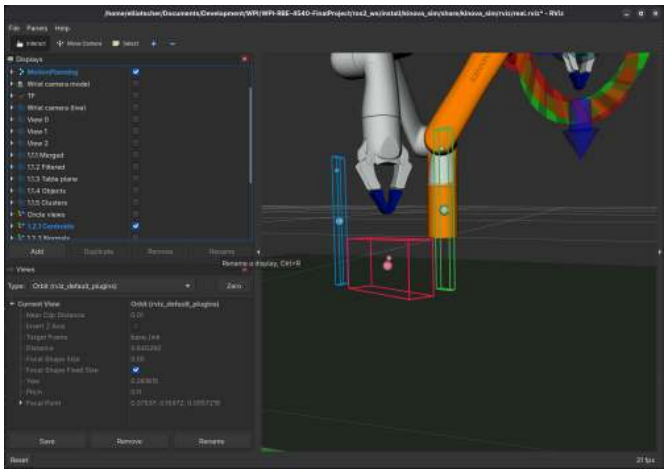
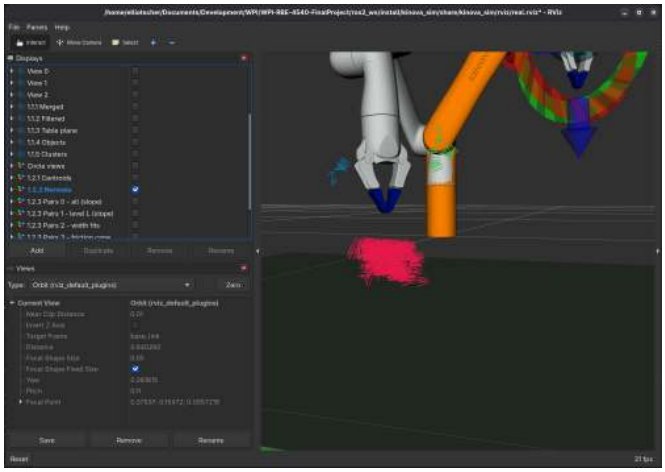
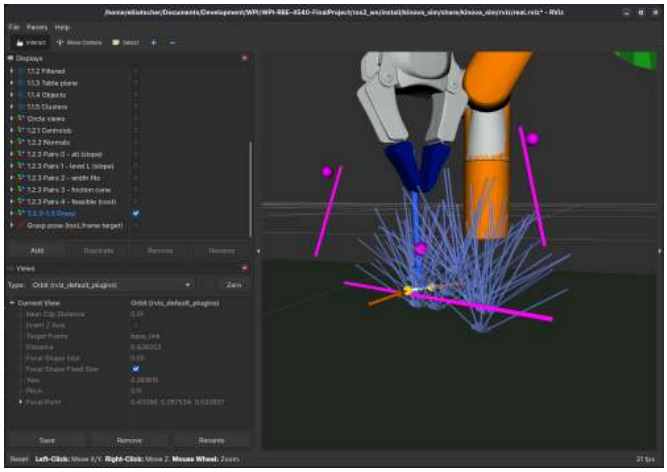














Videos of the real robot capturing the views, moving to the pre-grasp pose, grasping and lifting a cube (RBE_4540_Final_Cube_Compressed.mov) and a rectangular box (RBE_4540_Rectangle_Compressed.mov) are submitted alongside this report, and were shown in the in-person demonstration on 10/08/2026.